Reviewer Pack — Sharply-Bounded Refutability Vanishes at 3-SAT Threshold

Entry 08881f411f0c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 18:10:59 UTC

Sharply-Bounded Refutability Vanishes at 3-SAT Threshold

Entry ID: `08881f411f0c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 18:10:59 UTC

1. Conjecture

Field A (mathematical branch): Bounded arithmetic via the Cook–Reckhow–Krajicek correspondence: Σ^b_0 / PV-equational provability of CNF unsatisfiability, instantiated propositionally as the iterated unit-propagation + depth-1 failed-literal closure operator UPC (the rule fragment that captures polynomial-time PV recursion without nontrivial case analysis, distinct from constant-width resolution and from RUP-style proofs studied in SAT-solving) **Field B** (complexity object): Random unsatisfiable 3-CNF formulas at the SAT/UNSAT threshold; specifically the Boolean predicate UPC(F)∈{0,1} indicating whether the closure derives the empty clause without branching

Statement:

> Let F be a uniformly random 3-CNF on n=12 variables with m=⌊4.27·n⌋=51 clauses, conditioned on UNSAT (verified by exhaustive DPLL). Define UPC(F)=1 iff the following procedure terminates with ⊥: repeatedly (a) propagate any unit clause, then (b) for each currently unassigned variable x, tentatively assign x=0 and run pure unit-propagation; if conflict, permanently set x=1 (symmetrically for x=1); halt when no progress is possible or ⊥ is derived. Conjecture: Pr[UPC(F)=1] ≤ 0.20 over the conditioned-UNSAT distribution, and the empirical mean over ≥200 UNSAT samples will lie below 0.20 by at least 3 standard errors.

Rationale (proposer's reasoning):

> The Cook–Reckhow–Krajicek dictionary identifies sharply-bounded Σ^b_0/PV reasoning with rules that lack genuine case analysis—propositionally, exactly unit propagation plus depth-1 failed-literal probing. Random 3-CNFs at the satisfiability threshold are conjectured to encode expanding implication structure that is provably hard for any constant-width or branching-free system, matching the known intuition that PV cannot prove most random combinatorial UNSAT statements. Quantifying the UPC success rate at small n exposes the BA/proof-system gap empirically.

Taxonomy category: `BOUNDED_ARITHMETIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b16f7ab9bb0222a1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across ≥200 UNSAT-conditioned random 3-CNFs (n=12, m=51), let p̂ = empirical Pr[UPC(F)=1] with binomial SE σ=√(p̂(1-p̂)/N). SUPPORTED iff p̂ ≤ 0.20 AND (0.20 - p̂) ≥ 3σ. FALSIFIED iff p̂ > 0.20 AND (p̂ - 0.20) ≥ 3σ. Otherwise inconclusive.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.92	SAFE	SAFE
KARP_LIPTON	SAFE	0.97	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- random 3-SAT unsatisfiability threshold unit propagation failed literal - bounded arithmetic PV propositional proof complexity unit propagation closure - failed literal probe complete random 3-CNF refutation probability threshold`

Top relevant hits considered: - [http://arxiv.org/abs/1008.1260v1] Structure of random r-SAT below the pure literal threshold - [http://arxiv.org/abs/2405.16149v3] Small unsatisfiable k-CNFs with bounded literal occurrence - [http://arxiv.org/abs/1908.05361v2] Placing quantified variants of 3-SAT and Not-All-Equal 3-SAT in the polynomial hierarchy - [http://arxiv.org/abs/1411.7087v6] Consistency proof of a fragment of PV with substitution in bounded arithmetic - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/cs/0304041v1] $P \neq NP$, propositional proof complexity, and resolution lower bounds for the weak pigeonhole principle - [http://arxiv.org/abs/1804.05230v1] The threshold for SDP-refutation of random regular NAE-3SAT - [http://arxiv.org/abs/math/0410336v2] The critical probability for random Voronoi percolation in the plane is 1/2

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import json

def generate_3cnf(n, m):
    cnf = set()
    while len(cnf) < m:
        clause = tuple(sorted(random.sample(range(-n, 0), 1) + random.sample(range(1, n+1), 2)))
        if len(clause) == 3 and all(abs(lit) not in clause for lit in cnf):
            cnf.add(clause)
    return cnf

def dpll(cnf, assignment):
    unit_clauses = [lit for lit in cnf if sum(abs(lit) in assignment.values() for l in lit) == 1]
```

```
    while unit_clauses:
        lit = random.choice(unit_clauses)
        val = -assignment[lit[0]] if lit[0] in assignment else None
        if val is not None:
            assignment[lit[0]] = val
            cnf.discard(lit)
            unit_clauses = [l for l in cnf if sum(abs(lit) in assignment.values() for l in l) == 1
        else:
            return False, None
    return True, assignment

def upc(cnf):
    assignment = {}
    while True:
        success, _ = dpll(cnf, assignment)
        if not success:
            return False
        progress = False
        for x in range(1, 13):
            if x not in assignment:
                assignment[x] = 0
                new_assignment, _ = dpll(cnf, assignment)
                if not new_assignment:
                    assignment[x] = 1
                else:
                    progress = True
        for x in range(-12, -1):
            if -x not in assignment:
                assignment[-x] = 0
                new_assignment, _ = dpll(cnf, assignment)
                if not new_assignment:
                    assignment[-x] = 1
                else:
                    progress = True
        if not progress:
            return True

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 12
    m = 51
    cnf = generate_3cnf(n, m)
    unsat_cnf = []
    for _ in range(200):
        if not dpll(cnf, {})[0]:
            unsat_cnf.append(cnf.copy())
    upc_values = [upc(cnf) for cnf in unsat_cnf]
    p_hat = sum(upc_values) / len(upc_values)
    se = (p_hat * (1 - p_hat) / len(upc_values)) ** 0.5
    conjecture_holds = p_hat <= 0.20 and (0.20 - p_hat) >= 3 * se
    counterexample = "" if conjecture_holds else "UPC(F)=1 exceeded 0.20 by ≥3σ"
    return {
        "metric_name": "Pr[UPC(F)=1]",
        "metric_value": p_hat,
        "instances_tested": len(upc_values),
        "conjecture_holds": conjecture_holds,
```

```
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    p_hats = [r["metric_value"] for r in results if r["instances_tested"] > 0]
    se = (sum(p * (1 - p) for p in p_hats) / len(p_hats)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(p_hats)/len(p_hats):.4f} std={se:.4f} support_fraction
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(p_hats)/len(p_hats):.4f} std={se:.4f} support_fraction
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"UPC(F)=1 exceeded 0.20 by ≥3σ\" first_failing_s
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_136d91a0.py", line 91, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_136d91a0.py", line 69, in run_trial
    cnf = generate_3cnf(n, m)
          ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_136d91a0.py", line 22, in generate_3cnf
    if len(clause) == 3 and all(abs(lit) not in clause for lit in cnf):
                            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_136d91a0.py", line 22, in <genexpr>
    if len(clause) == 3 and all(abs(lit) not in clause for lit in cnf):
                                ^^^^^^^^
TypeError: bad operand type for abs(): 'tuple'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a TypeError in generate_3cnf (abs() applied to a tuple) before any trials ran, so no empirical data was produced and the pre-registered support condition cannot be evaluated. | next: Fix the CNF generator: iterate literals from each clause tuple (e.g., `all(abs(lit) not in {abs(l) for l in clause} for cl in cnf for lit in cl)`) and rerun with N≥200 UNSAT-conditioned samples.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	208991
2	preregistration	claude_max	opus	0	0	5459
3	novelty	claude_max	opus	0	0	3409
4	novelty	claude_max	opus	0	0	9402
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13201
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12045
7	judge	claude_max	opus	0	0	5005

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 257512 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/08881f411f0c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/08881f411f0c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/08881f411f0c.tar.gz` (if generated)